QuestionOS 技术可行性报告

项目: QuestionOS - 认知协同 Agent

任务: PRD 分析与技术可行性评估

评估人: 二号码农

日期: 2026-02-26

一、项目概述

QuestionOS 是一个认知协同 Agent，核心定位是"问题校准器"——通过多轮结构化对话帮助用户校准问题本质，而非提供解决方案。

核心价值主张:

- 错误的问题比错误的答案更危险
- 清晰度 > 速度
- 校准 > 行动
- 结构 > 情绪

二、技术架构建议

2.1 整体架构

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                                Frontend (Web)                                 │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐           │
│  │ 首页     │ │ 校准页   │ │ 报告页   │ │ 历史页   │ │ 账户页   │           │
│  └──────────┘ └──────────┘ └──────────┘ └──────────┘ └──────────┘           │
└──────────────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                                API Gateway                                    │
│                (认证、限流、路由、日志)                                        │
└──────────────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                             Backend Services                                 │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐           │
│  │ 用户服务         │  │ 会话服务         │  │ 校准服务         │           │
│  │ Auth/Profile     │  │ Conversation     │  │ Calibration      │           │
│  └──────────────────┘  └──────────────────┘  └──────────────────┘           │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐           │
│  │ UDPP服务         │  │ 报告服务         │  │ AI编排服务       │           │
│  │ User Profile     │  │ Report Gen       │  │ AI Orchestra     │           │
│  └──────────────────┘  └──────────────────┘  └──────────────────┘           │
└──────────────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                                Data Layer                                     │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐           │
│  │ PostgreSQL       │  │ Redis            │  │ Vector DB        │           │
│  │ 主数据存储       │  │ 会话缓存         │  │ 语义检索         │           │
│  └──────────────────┘  └──────────────────┘  └──────────────────┘           │
└──────────────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                                AI Layer                                       │
```

```
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐          │
│  │ 主模型           │  │ 降级模型         │  │ Embedding        │          │
│  │ GPT-4/Claude     │  │ GPT-3.5等        │  │ 文本向量化       │          │
│  └──────────────────┘  └──────────────────┘  └──────────────────┘          │
└──────────────────────────────────────────────────────────────────────────────┘
```

2.2 技术栈建议

层级	技术选型	理由
前端	Next.js 14 + TypeScript + Tailwind	SSR、TypeScript支持、组件生态丰富、部署方便
状态管理	Zustand / Jotai	轻量、简洁、适合对话场景
后端	Python FastAPI 或 Node.js NestJS	FastAPI对AI生态友好，NestJS结构清晰
数据库	PostgreSQL + Prisma/SQLAlchemy	关系型数据，成熟稳定，UDPP结构适合关系存储
缓存	Redis	会话状态、限流、缓存
向量库	pgvector 或 Pinecone	语义检索、历史问题相似度匹配
AI网关	自建AI编排层	成本控制、模型切换、Prompt管理
主模型	GPT-4.1 / Claude 3.5 Sonnet	高推理能力，校准质量保障
降级模型	GPT-3.5-turbo / DeepSeek	简单轮次降级，成本控制
Embedding	text-embedding-3-small	语义向量，检索支持
认证	NextAuth.js + JWT	OAuth(Gmail)、邮箱验证码
部署	Vercel(前端) + Railway/Fly.io(后端)	快速部署、自动扩缩
监控	LangSmith / LangFuse	AI调用追踪、成本监控

三、核心功能技术实现

3.1 问题结构识别

技术方案：

代码块

```
class QuestionStructureAnalyzer:
    """问题结构识别器"""

    STRUCTURE_TYPES = [
        "战略决策", "执行问题", "资源配置",
        "心智模式", "情绪干扰", "混合型"
    ]

    async def analyze(self, question: str, context: ConversationContext) -> StructureResult:
        # 1. 问题预处理（清洗、分词、实体提取）
        # 2. 结构分类（主模型推理）
        # 3. 核心变量提取
        # 4. 认知偏差识别
        # 5. 清晰度评分
        pass
```

关键指标：

- 结构分类准确率 ≥ 85%
- 偏差识别准确率 ≥ 80%
- 单次分析延迟 < 3s

3.2 多轮校准对话

技术方案：

代码块

```
class CalibrationSession:
    """校准会话管理"""

    async def process_turn(self, user_input: str) -> CalibrationResponse:
        # 1. 加载用户UDPP画像
        udpp = await self.load_udpp(user_id)

        # 2. 获取历史摘要
        history = await self.get_session_summary(session_id)

        # 3. 结构分析
        structure = await self.analyzer.analyze(user_input, history)

        # 4. 生成追问（核心！）
        questions = await self.generate_probing_questions(structure, udpp)

        # 5. 更新UDPP
        await self.update_udpp(user_id, structure)

        # 6. 计算清晰度
        clarity_score = self.calculate_clarity(structure, history)

        # 7. 判断是否达到报告阈值
        if clarity_score >= THRESHOLD:
            return await self.generate_calibration_report()

        return CalibrationResponse(
            questions=questions,
            clarity_progress=clarity_score,
            structure_visualization=structure.to_tree()
        )
```

追问生成策略（Prompt工程核心）：

代码块

```
你是问题校准专家。你的任务是生成追问，帮助用户理清问题本质。

核心原则：
1. 不提供解决方案，只提问
2. 每次追问不超过3个问题
```

```
3. 问题要有层级递进性
4. 识别并标注认知偏差
5. 引导用户自我发现

禁止行为:
- 直接给建议
- 提供行动方案
- 表达个人观点
- 使用"你应该"句式

输出格式:
{
  "questions": ["问题1", "问题2", "问题3"],
  "detected_biases": ["确认偏误", "沉没成本谬误"],
  "clarity_score": 0.65,
  "structure_update": {...}
}
```

3.3 UDPP 用户认知画像

数据模型:

代码块

```
-- 用户认知画像表
CREATE TABLE user_decision_profiles (
    user_id UUID PRIMARY KEY,

    -- 问题类型分布
    question_type_distribution JSONB DEFAULT '{}',
    -- 例: {"战略": 0.3, "执行": 0.4, "资源": 0.2, "心智": 0.1}

    -- 高频认知偏差统计
    cognitive_biases JSONB DEFAULT '{}',
    -- 例: {"确认偏误": 15, "沉没成本": 8, "锚定效应": 5}

    -- 清晰度演进历史(时间序列)
    clarity_history JSONB DEFAULT '[]',
    -- 例: [{"date": "2026-02-26", "score": 0.65}, ...]

    -- 决策稳定性指标
    decision_stability FLOAT DEFAULT 0.0,

```

```sql
    -- 最近会话摘要
    recent_sessions_summary TEXT,

    created_at TIMESTAMP,
    updated_at TIMESTAMP
);

-- 会话记录表
CREATE TABLE calibration_sessions (
    session_id UUID PRIMARY KEY,
    user_id UUID REFERENCES user_decision_profiles(user_id),

    -- 原始问题
    original_question TEXT NOT NULL,

    -- 校准后问题
    calibrated_question TEXT,

    -- 对话轮次
    turn_count INT DEFAULT 0,

    -- 结构分析结果
    structure_result JSONB,

    -- 最终清晰度
    final_clarity_score FLOAT,

    -- 是否生成报告
    report_generated BOOLEAN DEFAULT FALSE,

    created_at TIMESTAMP,
    completed_at TIMESTAMP
);

-- 对话轮次表
CREATE TABLE conversation_turns (
    turn_id UUID PRIMARY KEY,
    session_id UUID REFERENCES calibration_sessions(session_id),
    turn_number INT NOT NULL,

    user_input TEXT NOT NULL,
    ai_response JSONB NOT NULL,

    -- 该轮分析结果
    structure_delta JSONB,
    clarity_delta FLOAT,

```

```
    created_at TIMESTAMP
);
```

3.4 Calibration Report 生成

报告结构：

代码块

```
class CalibrationReport:
    """校准报告"""

    # 1. 问题源头追溯树
    question_tree: QuestionTree

    # 2. 核心变量识别
    core_variables: List[Variable]
    # 例: [{"name": "职业方向", "type": "外部", "weight": 0.8}]

    # 3. 认知偏差标注
    cognitive_biases: List[BiasAnnotation]
    # 例: [{"bias": "确认偏误", "evidence": "...", "impact": "high"}]

    # 4. 清晰度演进曲线
    clarity_evolution: List[ClarityPoint]

    # 5. 结构层级可视化数据
    structure_visualization: TreeData

    # 6. 后续追问建议（可选）
    follow_up_questions: List[str]
```

3.5 认知进步面板

技术实现：

- 数据聚合：定时任务计算用户指标
- 可视化：前端图表库（Recharts / ECharts）

- 阈值解锁：达到一定会话数后展示

四、技术风险与难点

4.1 高风险项

风险	影响	缓解措施
模型成本过高	用户每次对话多轮，成本压力大	1. 动态模型降级 2. 缓存相似问题 3. Prompt优化减少token
校准质量不稳定	用户满意度直接受影响	1. 建立评测数据集 2. 人工抽检机制 3. A/B测试迭代
追问过于机械	用户体验差，流失率高	1. 多样化Prompt模板 2. 个性化追问策略 3. 用户反馈机制

4.2 中风险项

风险	影响	缓解措施
UDPP建模复杂度高	开发周期长	1. MVP简化模型 2. 迭代优化
多轮对话上下文管理	token消耗大，易丢失	1. 摘要压缩技术 2. 关键信息提取
报告生成质量	用户价值感知弱	1. 模板化报告结构 2. 可视化增强

4.3 技术难点

1. **追问生成策略**
   - 难点：如何在"不给建议"前提下有效引导
   - 方案：大量高质量Prompt工程 + 评测数据集

2. **清晰度量化**
   - 难点：主观概念如何客观量化
   - 方案：多维度评分 + 加权聚合 + 人工校准

3. **认知偏差识别**
   - 难点：偏差类型多，识别准确率要求高
   - 方案：Few-shot示例 + 专用检测Prompt

五、开发计划建议

5.1 MVP范围（4-6周）

阶段	周期	交付物
Week 1-2	技术架构搭建	数据库设计、API骨架、认证系统
Week 2-3	核心AI流程	问题结构识别、追问生成、会话管理
Week 3-4	前端开发	首页、校准页、基础报告页
Week 4-5	UDPP基础版	用户画像存储、简单指标计算

Week 5-6	联调测试	端到端流程、bug修复、性能优化

5.2 MVP功能清单

必须有：

- [x] ~~邮箱注册/登录~~
- [x] ~~问题输入首页~~
- [x] ~~多轮校准对话（至少3轮）~~
- [x] ~~基础校准报告~~
- [x] ~~历史会话列表~~
- [x] ~~简单UDPP展示~~

暂不做：

- [ ] Gmail OAuth（后续增加）
- [ ] 认知进步面板（解锁机制）
- [ ] 高级可视化报告
- [ ] 复杂UDPP指标

六、成本估算

6.1 AI调用成本（MVP阶段）

假设：

- 每用户每会话平均5轮对话
- 每轮平均输入500 token，输出300 token
- 主模型使用率80%，降级模型20%

月成本估算（1000 DAU）：

```
日调用量 = 1000用户 × 1会话 × 5轮 = 5000次
月调用量 = 5000 × 30 = 150,000次

主模型成本:
- 输入: 150,000 × 0.8 × 500 × $0.03/1K = $1,800
- 输出: 150,000 × 0.8 × 300 × $0.06/1K = $2,160

降级模型成本:
- 输入: 150,000 × 0.2 × 500 × $0.001/1K = $15
- 输出: 150,000 × 0.2 × 300 × $0.002/1K = $18

月AI成本 ≈ $4,000 (1000 DAU)
```

6.2 基础设施成本

项目	月成本
Vercel Pro	$20
Railway/Fly.io	$50-100
PostgreSQL (Supabase/Railway)	$25
Redis	$10
域名 + SSL	$10
合计	$115-165

七、技术可行性结论

7.1 总体评估

维度	评分	说明
技术可实现性	⭐⭐⭐⭐⭐	核心技术成熟，无突破性难点
开发周期	⭐⭐⭐⭐	MVP 4-6周可行
成本可控性	⭐⭐⭐	AI成本需重点关注，需要降级策略
团队能力要求	⭐⭐⭐⭐	需要AI工程能力、全栈开发

7.2 建议优先级

1. **P0 - 必须实现：**
    - 多轮校准对话核心流程
    - 基础问题结构识别
    - 简单追问生成
    - 校准报告输出

2. **P1 - 重要但可延后：**
    - UDPP用户画像
    - 认知进步面板
    - Gmail OAuth

3. **P2 - 可选优化：**
    - 高级可视化
    - 复杂偏差识别
    - 社区功能

7.3 风险提示

1. **成本风险**：多轮对话成本高，需要严格的降级策略
2. **质量风险**：追问质量决定用户留存，需要大量评测
3. **差异化风险**：竞品可能快速跟进，需要建立数据壁垒

八、下一步行动建议

1. **立即启动**：
   - 确定技术栈选型
   - 设计数据库Schema
   - 搭建开发环境

2. **本周完成**：
   - 核心Prompt模板设计
   - AI调用架构设计
   - 前端框架搭建

3. **下周目标**：
   - 完成问题结构识别POC
   - 完成基础对话流程
   - 输出详细API设计文档

报告完成时间: 2026-02-26 12:30

评估人: 二号码农

状态: 已完成

"在行动之前，先校准问题。"